INDIVIDUAL PROJECT REPORT

Review 2 — Design, Development & Progress (14/9/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role
Laksh Ahuja	24BDS0212	_____	Central server, dashboard, network QoS, cyclone simulation and testing

Team: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417). **Repository:** https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Summary

Since Review 1 I designed and built the central prediction server of our prototype: the UDP listener, per-node storage, risk scoring, the REST API and a live dashboard, and I worked on the team's Packet Tracer topology. The prototype runs end to end with my teammates' sensor nodes and gateway. This report shows the design, the working code, why I chose this design, the updated feasibility, the problems I hit and how I solved them, and my contribution.

1. Detailed Design

Input → process → output: **input** is a JSON record forwarded by the gateway over UDP port 9002 (node, seq, time, wind, temp, pressure, traffic_class); **process** checks the sequence number, fills gaps using the team's interpolation function, computes a risk score and stores the record; **output** is three REST endpoints and an HTML dashboard (Figure 1).

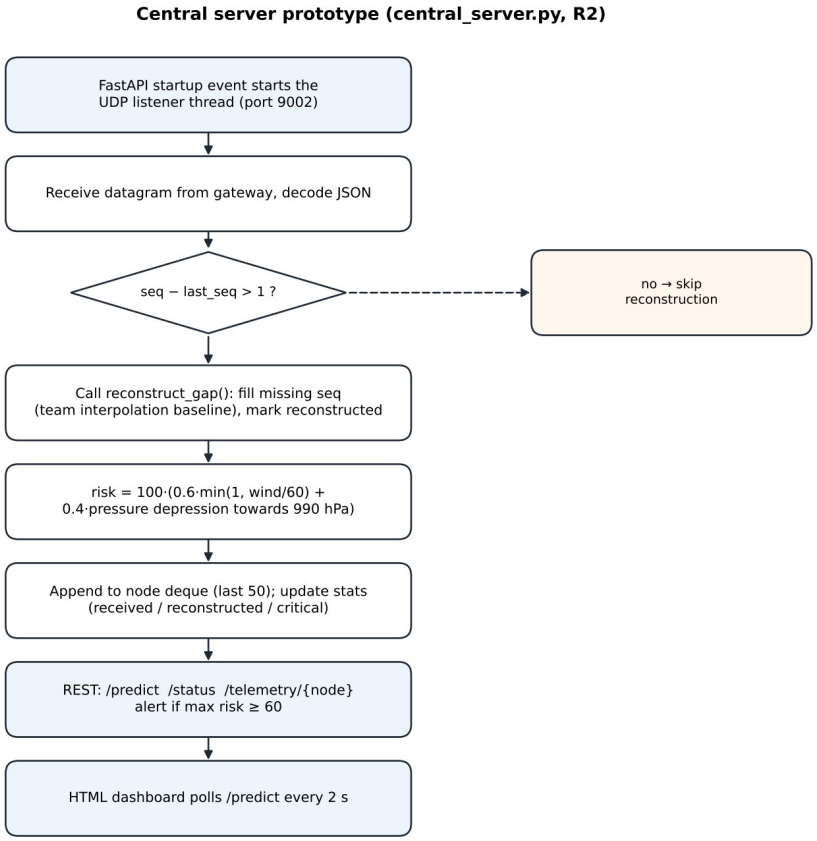


Figure 1: Flowchart of my central server (prototype)

Table 1: Labelled parameters of my module

Parameter	Value	Meaning
GATEWAY_TO_SERVER_PORT	9002 (UDP)	Where the gateway sends packets
HISTORY_LEN	50	Readings kept per node (deque)
WIND_RISK_THRESHOLD	60 km/h	Wind at which the wind part of risk saturates
PRESSURE_RISK_THRESHOLD	990 hPa	Pressure at which the pressure part saturates
Risk formula	100·(0.6·w + 0.4·p)	w = min(1, wind/60); p = depression from 1013 towards 990 hPa
Alert rule	max node risk ≥ 60	Shown in /predict and on the dashboard
Dashboard refresh	2 s polling	Fetches /predict

2. Actual Working Progress since Review 1

- **Code committed:** `central_server.py` (189 lines, including the `reconstruct_gap()` function written by Sreehari), in repository commit `82bcb8a`.

- **Running:** server, gateway and sensors run together; endpoints `/predict`, `/status`, `/telemetry/{node}` and the dashboard work.
- **Network:** the team's Packet Tracer topology is complete with OSPF routing; a ping from BUOY-01 to the prediction server succeeds (Figure 3).

The heart of the server is short; this is the startup fix and the prediction endpoint:

```
@app.on_event("startup")
def start_listener():
    threading.Thread(target=udp_listener, daemon=True).start()

@app.get("/predict")
def predict():
    with lock:
        latest = {node_id: dq[-1] for node_id, dq in telemetry_store.items() if dq}
    overall = max(rec["risk"] for rec in latest.values())
    return {"overall_risk": overall, "alert": overall >= 60, "nodes": latest}
```

To produce dated evidence for this report, the Review 2 prototype (repository commit `82bcb8a`, 16/9/2026) was re-run unchanged with three sensor nodes for 60 seconds; the logs are in `docs/reports/evidence/r2_prototype/`. The team's original 12-second test reported 66 packets received, 11 reconstructed and 2 critical, with overall risk 100 and the alert active.

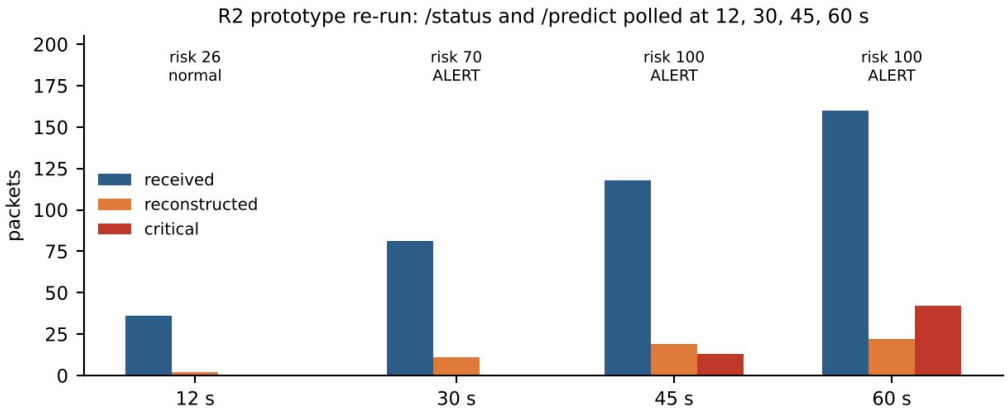


Figure 2: My server's /status and /predict during the prototype re-run

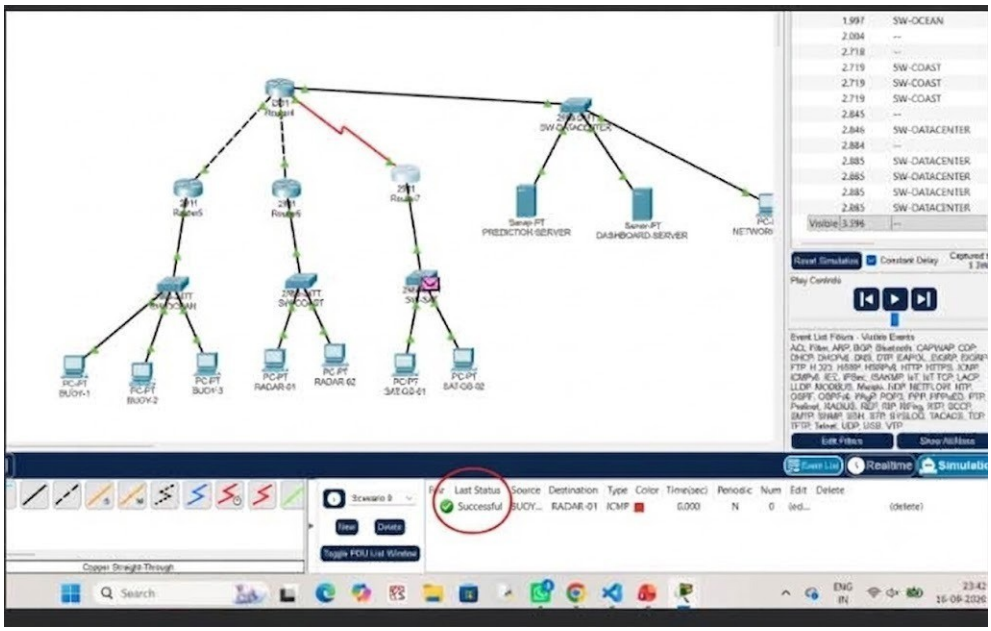


Figure 3: The team's Packet Tracer topology (joint work); the data-centre side hosts my prediction and dashboard servers

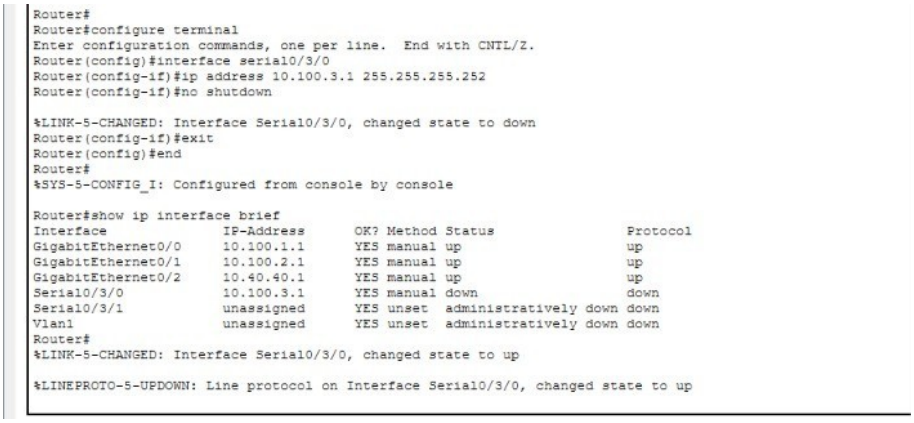


Figure 4: Core router configuration (show ip interface brief) linking the WAN to the data centre

Completion: about 40 % of my planned scope. **Done:** server pipeline, risk score, REST API, polling dashboard, topology. **Remaining:** priority scheduling on a bandwidth-limited link, a learned model for missing data (with Sreehari and Anvay), real-time updates over WebSocket, a realistic cyclone model, and automated tests.

Table 2: Test runs performed on my module

Test run	Setup	Observation
Team test, 12 s, 2 nodes	Server + gateway + BUOY_07, RADAR_A	66 packets, 11 reconstructed, 2 critical; risk 100, alert on
Startup test	Sensors started before the listener	/predict empty → fixed with startup event
Re-run, 60 s, 3 nodes	Prototype commit 82bcb8a	160 received, 22 reconstructed, 42 critical; alert at ≈30 s
Robustness test	One non-JSON datagram to port 9002	Listener thread died; /status stayed at 0 received
Packet Tracer	ping BUOY-01 → 10.40.40.10	Successful end to end over OSPF

3. Justification of Design Choices

Table 3: Comparison of alternatives for my module

Choice	Option	Cost	Ease of use	Fit for our problem	Decision
Web framework	Flask	Free	Very easy	Sync by default; background thread needs care	
	Django	Free	Heavy setup	Too large for a small API	
	FastAPI	Free	Easy	Async + startup hooks for the UDP thread; automatic API docs	Chosen
Transport	TCP	—	Easy	Retransmissions hide the loss we must study; adds delay	
	MQTT broker	Free	Extra service	Good later; adds a broker to run	
	UDP datagrams	—	Easy	Loss is visible and measurable; matches lossy links	Chosen
Dashboard update	**Polling 2 s**	—	Simplest	Good enough for the prototype	Chosen now
	WebSocket push	—	Moderate	Lower latency; planned for the final system	Next phase

4. Updated Feasibility

Table 4: Economic picture for my part

Aspect	Review 1 estimate	Now (Review 2)	Reason for change
Software cost	₹0	₹0	All tools open source
Server hosting	Own laptop	Own laptop; a small cloud VM later ≈ US$5–10 / month	Needed only for a public demo
Effort	3 weeks for prototype	On schedule	Startup race fixed in one day

Social: coastal and island communities often have the worst connectivity, so a server that keeps a usable picture despite missing data supports fairer disaster response. **Environmental:** no new hardware is needed for the server tier, and avoiding retransmissions saves energy at the remote end.

5. Problems Encountered and How I Adapted

Table 5: Problems in my module

Problem	How I diagnosed it	What I changed	Result
`/predict` returned empty data at first	Server log showed the listener starting after sensors had begun sending; the thread was started at import time	Start the UDP listener in FastAPI's startup event	Data appears immediately; fixed
Processes started from separate shells failed silently	Background processes lost their working directory, so imports failed	Launch the server, gateway and sensors from one script	All components start reliably; fixed
One malformed packet stopped the server receiving	Sent a non-JSON datagram during testing: the listener thread raised an exception and died; `/status` then showed 0 packets received	Input validation and exception handling planned for the next phase	Workaround: restart; fix scheduled
Gaps can only be repaired after the next packet	Found while reviewing the logs: a gap is known only when a later sequence number arrives	Plan real-time estimates for overdue packets	Design limitation recorded

Plan from Review 2 to Review 3

Week	Planned work
15–21 Sep	Token-bucket QoS scheduler with critical reserve; FIFO baseline for comparison
22–28 Sep	Server hardening: validation, late/duplicate packets, real-time estimates for overdue packets
29 Sep–5 Oct	WebSocket dashboard; physics-based cyclone model; one-command launcher
6–9 Oct	Automated test suite; measurements for the final report

6. Individual Contribution Evidence

Table 6: Division of work

Tasks I did	Tasks done by teammates
Central server: UDP listener, storage, risk score, alert rule	Anvay: edge gateway (classification, priority queue, alert throttling)
REST API (/predict, /status, /telemetry) and HTML dashboard	Sreehari: sensor-node simulator and the reconstruct_gap() interpolation used by my server
Startup race fix; single-script launch; end-to-end test runs	Team: shared record format (common.py), Packet Tracer topology

- Code: `central_server.py` at commit `82bcb8a` (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Logs: `server.log`, `api.log` and `crash_srv.log` in `docs/reports/evidence/r2_prototype/`.

> *[Insert here: screenshot of my own commits / work log and a photo or screen recording of me testing. Note: the team's early commits were pushed from one shared GitHub account, so the file-level split above is the record of who wrote what.]*

R2 Video Plan

Segment	What I show
Design walkthrough (2 min)	Face + screen; flowchart and parameter table
Live progress with my own work on screen (2 min)	Face + screen; code running and the logs
One problem and its solution (1 min)	Face + screen; the problem table and the evidence log
Updated cost and my contribution (1 min)	Face + screen; cost table and the division-of-work table